tf.compat.v1.distributions.Beta Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distributions/Beta

> *Beta distribution.*

Function Signatures

```
tf.compat.v1.distributions.Beta( concentration1=None,
concentration0=None, validate_args=False,
allow_nan_stats=True, name='Beta' )
```

(0, 1)

Code Examples

```
tf.compat.v1.distributions.Beta(
concentration1=None,
concentration0=None,
validate_args=False,
allow_nan_stats=True,
name='Beta'
)
```

```
tf.compat.v1.distributions.Beta(
concentration1=None,
concentration0=None,
validate_args=False,
allow_nan_stats=True,
name='Beta'
)
```

```
pdf(x; alpha, beta) = x**(alpha - 1) (1 - x)**(beta - 1) / Z
Z = Gamma(alpha) Gamma(beta) / Gamma(alpha + beta)
```


```
pdf(x; alpha, beta) = x**(alpha - 1) (1 - x)**(beta - 1) / Z
Z = Gamma(alpha) Gamma(beta) / Gamma(alpha + beta)
```


```
concentration1 = alpha = mean * total_concentration
concentration0 = beta  = (1. - mean) * total_concentration
```


```
concentration1 = alpha = mean * total_concentration
concentration0 = beta  = (1. - mean) * total_concentration
```


```
import tensorflow_probability as tfp
tfd = tfp.distributions
# Create a batch of three Beta distributions.
alpha = [1, 2, 3]
beta = [1, 2, 3]
dist = tfd.Beta(alpha, beta)
dist.sample([4, 5])  # Shape [4, 5, 3]
# `x` has three batch entries, each with two samples.
x = [[.1, .4, .5],
[.2, .3, .5]]
# Calculate the probability of each pair of samples under the
corresponding
# distribution in `dist`.
dist.prob(x)         # Shape [2, 3]
```

```
import tensorflow_probability as tfp
tfd = tfp.distributions
# Create a batch of three Beta distributions.
alpha = [1, 2, 3]
beta = [1, 2, 3]
dist = tfd.Beta(alpha, beta)
dist.sample([4, 5])  # Shape [4, 5, 3]
# `x` has three batch entries, each with two samples.
x = [[.1, .4, .5],
[.2, .3, .5]]
# Calculate the probability of each pair of samples under the
corresponding
# distribution in `dist`.
dist.prob(x)         # Shape [2, 3]
```

Documentation

Beta distribution.

Inherits From: Distribution

The Beta distribution is defined over the (0, 1) interval using parameters concentration1 (aka "alpha") and concentration0 (aka "beta").

Mathematical Details

The probability density function (pdf) is,

where:

- concentration1 = alpha,
- concentration0 = beta,
- Z is the normalization constant, and,

- Gamma is the gamma function.

The concentration parameters represent mean total counts of a 1 or a 0, i.e.,

where mean in (0, 1) and total_concentration is a positive real number representing a mean total_count = concentration1 + concentration0.

Distribution parameters are automatically broadcast in all functions; see examples for details.

Samples of this distribution are reparameterized (pathwise differentiable). The derivatives are computed using the approach described in (Figurnov et al., 2018).

Examples

Compute the gradients of samples w.r.t. the parameters:

References

Attributes

Stats return +/- infinity when it makes sense. E.g., the variance of a Cauchy distribution is infinity. However, sometimes the statistic is undefined, e.g., if a distribution's pdf does not achieve a maximum within the support of the distribution, the mode is undefined. If the mean is undefined, then by definition the variance is undefined. E.g. the mean for Student's T for df = 1 is undefined (no clear way to say it is either + or - infinity), so the variance = E[(X - mean)**2] is also undefined.

May be partially defined or unknown.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

May be partially defined or unknown.

Currently this is one of the static instances
distributions.FULLY_REPARAMETERIZED
or distributions.NOT_REPARAMETERIZED.

Methods

batch_shape_tensor

View source

Shape of a single sample from a single event index as a 1-D Tensor.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

View source

Cumulative distribution function.

Given random variable X, the cumulative distribution function cdf is:

Additional documentation from Beta:

View source

Creates a deep copy of the distribution.

covariance

View source

Covariance.

Covariance is (possibly) defined only for non-scalar-event distributions.

For example, for a length-k, vector-valued distribution, it is calculated as,

where Cov is a (batch of) k x k matrix, 0 <= (i, j) < k, and E denotes expectation.

Alternatively, for non-vector, multivariate distributions (e.g., matrix-valued, Wishart), Covariance shall return a (batch of) matrices under some vectorization of the events, i.e.,

where Cov is a (batch of) k' x k' matrices,
0 <= (i, j) < k' = reduce_prod(event_shape), and Vec is some function mapping indices of this distribution's event dimensions to indices of a length-k' vector.

cross_entropy

View source

Computes the (Shannon) cross entropy.

Denote this distribution (self) by P and the other distribution by Q. Assuming P, Q are absolutely continuous with respect to one another and permit densities p(x) dr(x) and q(x) dr(x), (Shanon) cross entropy is defined as:

where F denotes the support of the random variable X ~ P.

entropy

View source

Shannon entropy in nats.

event_shape_tensor

View source

Shape of a single sample from a single batch as a 1-D int32 Tensor.

is_scalar_batch

View source

Indicates that batch_shape == [].

is_scalar_event

View source

Indicates that event_shape == [].

kl_divergence

View source

Computes the Kullback--Leibler divergence.

Denote this distribution (self) by p and the other distribution by q. Assuming p, q are absolutely continuous with respect to reference measure r, the KL divergence is defined as:

where F denotes the support of the random variable X ~ p, H[., .] denotes (Shanon) cross entropy, and H[.] denotes (Shanon) entropy.

log_cdf

View source

Log cumulative distribution function.

Given random variable X, the cumulative distribution function cdf is:

Often, a numerical approximation can be used for log_cdf(x) that yields a more accurate answer than simply taking the logarithm of the cdf when x << -1.

Additional documentation from Beta:

log_prob

View source

Log probability density/mass function.

Additional documentation from Beta:

log_survival_function

View source

Log survival function.

Given random variable X, the survival function is defined:

Typically, different numerical approximations can be used for the log survival function, which are more accurate than 1 - cdf(x) when x >> 1.

View source

View source

Additional documentation from Beta:

param_shapes

View source

Shapes of parameters given the desired shape of a call to sample().

This is a class method that describes what key/value arguments are required to instantiate the given Distribution so that a particular shape is returned for that instance's call to sample().

Subclasses should override class method _param_shapes.

param_static_shapes

View source

param_shapes with static (i.e. TensorShape) shapes.

This is a class method that describes what key/value arguments are required to instantiate the given Distribution so that a particular shape is returned for that instance's call to sample(). Assumes that the sample's shape is known statically.

Subclasses should override class method _param_shapes to return constant-valued tensors when constant values are fed.

View source

Probability density/mass function.

Additional documentation from Beta:

quantile

View source

Quantile function. Aka "inverse cdf" or "percent point function".

Given random variable X and p in [0, 1], the quantile is:

sample

View source

Generate samples of the specified shape.

Note that a call to sample() without arguments will generate a single sample.

stddev

View source

Standard deviation.

Standard deviation is defined as,

where X is the random variable associated with this distribution, E